Code-Layout, Readability and Reusability

Date	03 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S. No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform 4-space indentation used throughout all Python files (app.py and all 5 Jupyter notebooks)	Yes	PEP8 standard followed in all .py and .ipynb files
2	Proper File Structure	Files and folders are logically organized into data/, templates/, notebooks, and model files at root	Yes	5 notebooks + app.py + templates/ + data/ — clear separation
3	Meaningful Variable Names	Variables clearly reflect their purpose — e.g., X_train, y_test, model_columns, input_dict, risk_score	Yes	Descriptive names used across all notebooks and app.py
4	Function / Method Names	Flask route functions are descriptively named: home(), predict(), batch_form(), batch_predict()	Yes	Function names clearly describe their HTTP route and purpose
5	Code Comments	Inline comments explain logic steps — e.g., # Build base input dict, # Fill one-hot encoded fields	Yes	Comments added in app.py and all 5 notebook cells
6	Modular Design	Code is split into 5 separate Jupyter notebooks by project phase plus app.py for the web application	Yes	Each notebook handles one Epic — clean separation of concerns

S. No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
7	No Redundant Code	Duplicate columns removed after one-hot encoding (drop_first=True). Raw DAYS_BIRTH and DAYS_EMPLOYED dropped after creating cleaned AGE_YEARS and YEARS_EMPLOYED features	Yes	5 redundant columns dropped in preprocessing notebook
8	Error Handling	Flask app uses if/else checks for prediction results. One-hot encoded column existence is verified before setting values (if occ_col in input_dict). CSV file parsing is wrapped	Partial	Basic error handling in app.py; try/except can be added in future

Reusable Components / Modules:

S. No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	credit_approval_model.pkl (XGBoost Model)	Python, XGBoost, Joblib	Loaded in app.py for both single prediction and batch CSV screening	High
2	model_columns.pkl (Feature Column List)	Python, Joblib	Used in app.py predict() and batch_predict() to ensure correct feature order	High
3	input_dict builder (Feature Engineering Logic)	Python, Pandas	Same logic reused in both /predict and /batch_predict routes in app.py	High
4	03_preprocessing.ipynb (Preprocessing Pipeline)	Python, Pandas, Scikit-learn	Saved X_train/X_test CSVs reused in 04_model_building.ipynb for all 4 models	High
5	watson_model_pipeline.pkl (Watson Deployment Pipeline)	Python, Scikit-learn, IBM Watson ML	Used in 05_watson_deployment.ipynb and can be reused for any future model upload	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	★★★★★ (5/5)	5 notebooks clearly separated by project phase. Project folder well-organized with data/, templates/, notebooks, and model files at root level.
Readability	★★★★★ (5/5)	Descriptive variable names (X_train, y_test, model_columns, input_dict), consistent 4-space indentation, inline comments throughout all files.
Reusability	★★★★★ (5/5)	Saved model (.pkl) and feature columns (.pkl) reused across both Flask routes. Preprocessing pipeline output reused directly in model building notebook.
Documentation / Comments	★★★★■ (4/5)	All 5 notebooks have cell-level comments explaining each step. app.py has inline comments. Formal docstrings can be added to functions for improvement.
Overall Score	**4.75 / 5**	The codebase is well-structured, readable, and reusable. Modular design across 5 notebooks and Flask app ensures maintainability. Minor improvement possible by adding try/except blocks and function docstrings.